

## IMPORT LIBRARIES

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier, export_graphviz
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import graphviz
```

## LOAD THE DATASET

```
In [2]: iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

In [3]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_
dt_clf = DecisionTreeClassifier(max_depth=None, random_state=42)
dt_clf.fit(X_train, y_train)
y_pred = dt_clf.predict(X_test)
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred))
```

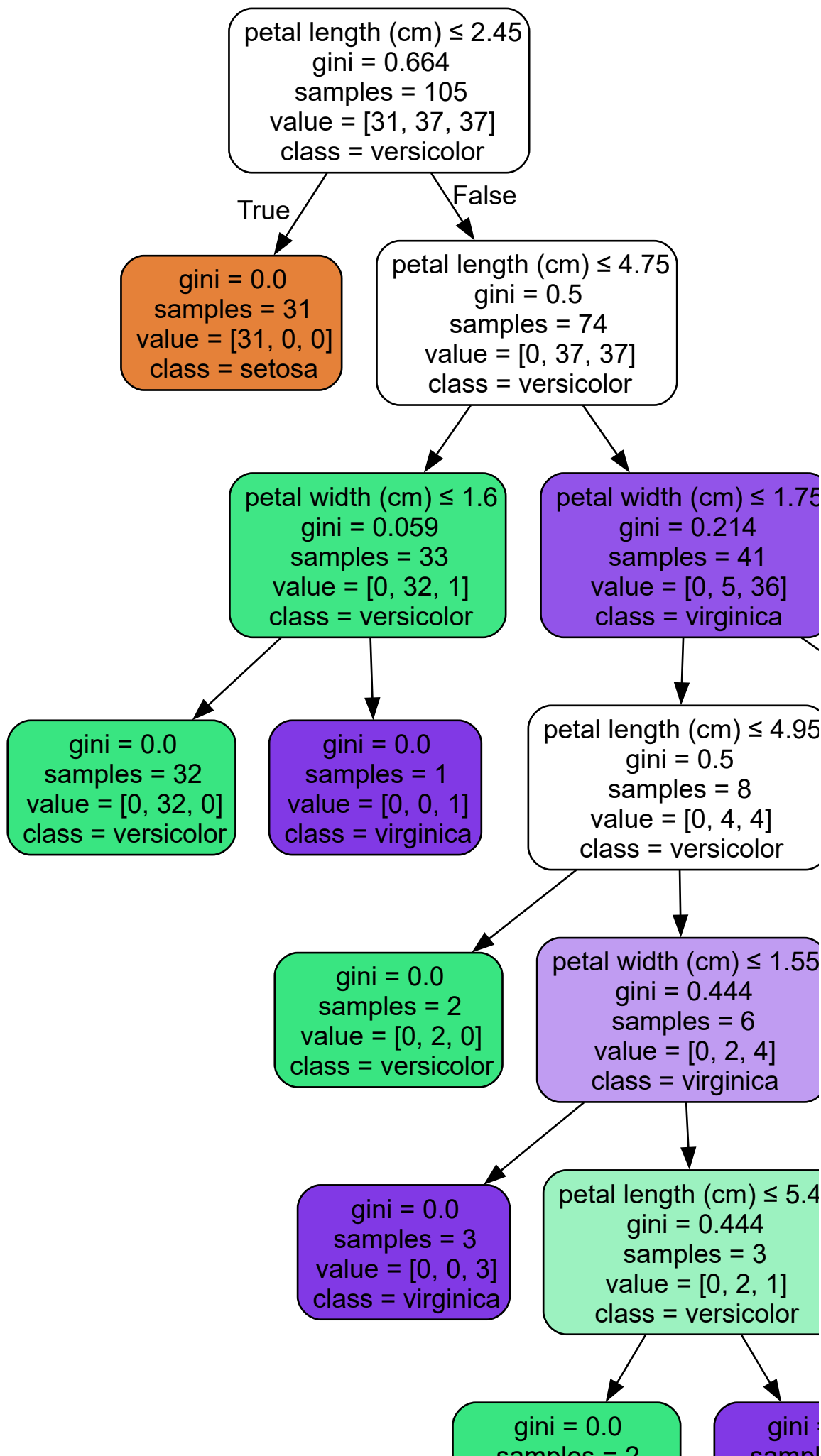
Decision Tree Accuracy: 1.0

## VISUALIZE THE TREE USING GRAPHVIZ

```
In [4]: dot_data = export_graphviz(dt_clf, out_file=None,
feature_names=feature_names,
class_names=target_names,
filled=True, rounded=True,
special_characters=True)

graph = graphviz.Source(dot_data)
graph.render("decision_tree")
graph
```

Out[4]:

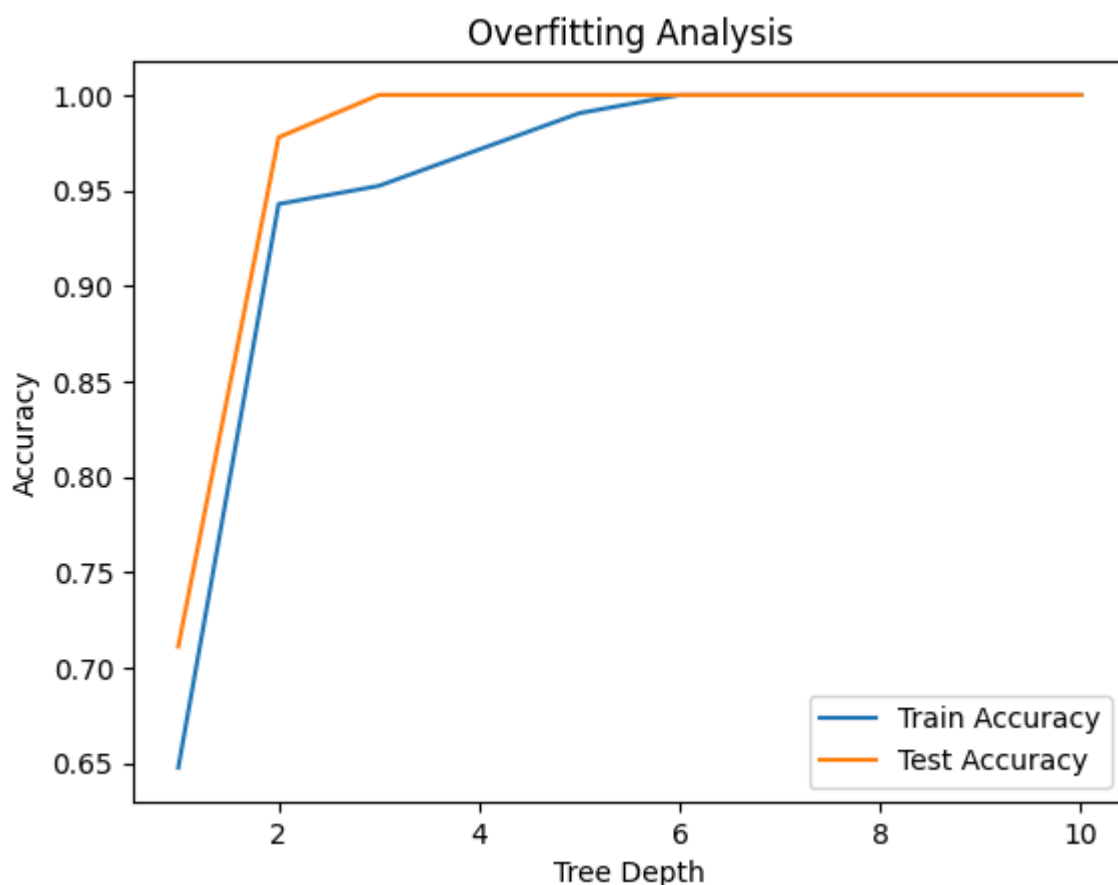


samples = 2  
value = [0, 2, 0]  
class = versicolor

samples = 2  
value = [0, 2, 0]  
class =

### ANALYZE THE OVERFITTING BY CONTROLLING THE DEPTH

```
In [7]: depths = range(1,11)
train_accuracies = []
test_accuracies = []
for depth in depths:
    clf = DecisionTreeClassifier(max_depth=depth, random_state=42)
    clf.fit(X_train, y_train)
    train_accuracies.append(clf.score(X_train, y_train))
    test_accuracies.append(clf.score(X_test, y_test))
plt.plot(depths, train_accuracies, label="Train Accuracy")
plt.plot(depths, test_accuracies, label="Test Accuracy")
plt.xlabel("Tree Depth")
plt.ylabel("Accuracy")
plt.title("Overfitting Analysis")
plt.legend()
plt.show()
```



### TRAIN RANDOM FOREST AND COMPARE ACCURACY

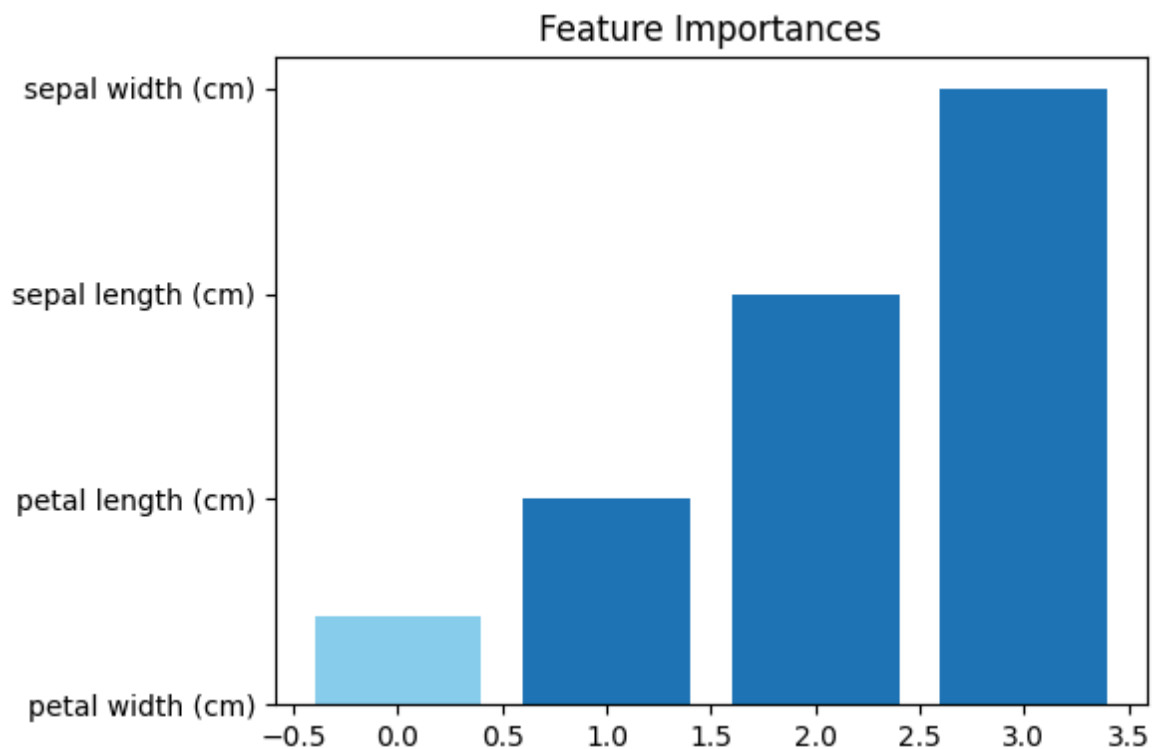
```
In [8]: rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rf_clf.fit(X_train, y_train)
y_pred_rf = rf_clf.predict(X_test)
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

Random Forest Accuracy: 1.0

## INTERPRET FEATURE IMPORTANCES

```
In [14]: importances = rf_clf.feature_importances_
indices = np.argsort(importances)[::-1]

#Plot
plt.figure(figsize=(6,4))
plt.title("Feature Importances")
plt.bar(range(X.shape[1]), importances[indices], color = "skyblue", align="center")
plt.bar(range(X.shape[1]), [feature_names[i] for i in indices])
plt.tight_layout()
plt.show()
```



## EVALUATE USING CROSS VALIDATION

```
In [15]: from sklearn.model_selection import cross_val_score
dt_scores = cross_val_score(dt_clf, X, y, cv=5)
rf_scores = cross_val_score(rf_clf, X, y, cv=5)

print("Decision Tree CV Accuracy:", dt_scores.mean())
print("Random Forest CV Accuracy:", rf_scores.mean())
```

Decision Tree CV Accuracy: 0.9533333333333334

Random Forest CV Accuracy: 0.9666666666666668

In [ ]: